```
$text = "The skiing in Turkey is very well-kept secret. \u{1F970} \u{1F3BF}
Kartalkaya \u{1F3BF} \u{1F970}";

var_dump(str_contains($text, "skiing in Turkey"));
var_dump(str_contains($text, "\u{1F3BF}"));
var_dump(str_starts_with($text, "The"));
var_dump(str_starts_with($text, "skiing"));
var_dump(str_ends_with($text, "\u{1F970}"));

Output:
bool(true)
bool(true)
bool(true)
bool(false)
bool(true)
```

Note that we have multi-byte characters in the example code (as Unicode escapes) to demonstrate that although the three functions are not character aware *per se*, they can be safely used with multi-byte data: the byte-level comparison either finds the sub-string or it does not. All three are also binary safe; the following demonstrates that `str_contains()` will read beyond a null byte:

```
$text = "Let's go \x00 skiing!";

var_dump(str_contains($text, "go \x00 skiing"));
var_dump(str_contains($text, "go \x00 snowboarding"));

Output:
bool(true)
bool(false)
```

`strpos()`, `mb_strpos()`, et al.

If it is necessary to know where in a string a given sub-string can be found, there are several related functions for finding the answer: `strpos()` (and related case-insensitive/reversed forms) and the multi-byte equivalent forms `mb_strpos()` et al.

```
$quote = 'We demand rigidly defined areas of doubt and uncertainty!';

var_dump(strpos($quote, 'We demand'));
var_dump(stripos($quote, 'wE dEmAnD'));
```